ResQRoute: A Risk- and Resource-Aware Adaptive Inference Controller for Post-Disaster Building Damage Assessment

[Author 1][1], [Author 2][2], [Author 3][3]

Department of Computer Science and Engineering, [Institution Name], [City], [Country]

Abstract—Rapid assessment of building damage from satellite imagery is critical after floods, hurricanes and wildfires, but running a large deep network on every building is slow, energy-hungry and often impossible on field devices with intermittent connectivity. Conventional model cascades escalate an input to a larger model whenever the small model's confidence falls below a fixed threshold, ignoring both the device's current resources and the very different consequences of different errors. This paper proposes ResQRoute, a learned routing controller that, for each building, chooses among five actions: accept a lightweight prediction, escalate to one of two larger models, defer to a priority queue, or flag for human review. The decision jointly uses prediction uncertainty, cheap image-quality cues, the device's latency budget, memory, network and battery state, and an asymmetric penalty that weights missed severe damage most heavily. Three ImageNet-pretrained classifiers (MobileNetV3-Small, ResNet-18 and EfficientNet-B0) were fine-tuned on 6,000 balanced pre/post building crops from three disasters in the xBD dataset, and their predictions, CPU latency and memory were logged for every crop. On a tile-disjoint test set evaluated under 6,816 simulated device states, the controller reached a macro-F1 of 0.631 at an average of 27 ms per image, matching the 0.610 of always running EfficientNet-B0 (126 ms) or ResNet-18 (235 ms) at roughly one-fifth to one-ninth of the computation, while exceeding the device budget in only 0.5% of decisions versus 33% for a fixed-confidence cascade and reducing dangerous severe-to-no-damage errors from 7.4% to 4.6%.

Keywords—*Adaptive inference; model cascade; building damage assessment; xBD; edge computing; cost-sensitive learning; learning to defer; disaster response*

I. INTRODUCTION

After a major natural disaster, emergency managers must decide within hours where to send search-and-rescue teams, engineers and supplies. Very-high-resolution satellite imagery makes it possible to assess thousands of buildings remotely [1], and public benchmarks such as xBD [2], [3] have enabled convolutional neural networks to grade building damage automatically. In practice, however, these assessments are often run on laptops or edge devices close to the affected area, where computation, memory, battery and network bandwidth are scarce and change over time [4].

Not every building is equally hard to assess. A clearly intact house in a sharp image can be classified correctly by a lightweight network, whereas a partially collapsed roof under smoke or shadow may require a stronger model or a human analyst. Model cascades exploit this by running a cheap model first and escalating only uncertain inputs [5], [6], [7]. Most cascades, though, use a single fixed confidence threshold. They do not consider whether the device can currently afford the larger model, and they treat all errors equally, although predicting *no damage* for a destroyed building is far more dangerous than confusing *minor* with *no damage*.

This paper addresses the question: **which model should process each image under the current resource and safety constraints?** The contributions are:

1) **A five-action routing controller** (accept, escalate to either of two larger models, defer, human review) trained as a supervised classifier on oracle routes derived from the logged behaviour of three base models.
2) **Joint decision features** combining prediction uncertainty, image-quality cues, and the device state (latency budget, free memory, network, battery, queue length), together with an **asymmetric safety weighting** that penalises missed severe damage most heavily.
3) **A deferred-escalation queue** ordered by a priority score $P = U \times R \times D \times A$, so that postponed, uncertain and potentially dangerous buildings are processed first when resources return.
4) **A reproducible, laptop-scale evaluation** on xBD with tile-disjoint splits, real CPU latency and memory measurements, five baselines, and bootstrap confidence intervals.

Section II surveys related work, Section III describes the proposed method, Section IV the experimental setup, Section V the results, Section VI the limitations, and Section VII concludes.

II. LITERATURE SURVEY

A. Building damage assessment from remote sensing

Early damage mapping relied on change detection between pre- and post-event imagery and on manual photo-interpretation; Dong and Shan [1] review these techniques for earthquakes. The xBD dataset [2], [3] provides about 850,000 building polygons over 19 events with a four-level damage scale (no, minor, major damage, destroyed) and was released with the xView2 challenge. Deep-learning approaches on xBD include siamese and multi-temporal fusion CNNs [8], [9], joint segmentation and damage classification (RescueNet) [10], and attention-based multiscale networks such as BDANet [11]. These works aim to maximise accuracy with a single, fixed model; none of them adapts the amount of computation per building or to the deployment device.

B. Efficient convolutional networks

Mobile-oriented architectures reduce inference cost: MobileNetV3 [12] combines depthwise-separable convolutions with architecture search, ResNet [13] introduced residual learning and remains a common medium-sized backbone, and EfficientNet [14] scales depth, width and resolution jointly. All three are typically initialised from ImageNet [15]. Choosing one of them fixes the accuracy-cost trade-off for every input; ResQRoute instead uses all three as interchangeable experts.

C. Adaptive and conditional inference

Cascades date back to the Viola-Jones detector [5]. In deep learning, early-exit networks such as BranchyNet [16], MSDNet [17] and Shallow-Deep Networks [18] attach classifiers to intermediate layers and stop when confident; SkipNet [19] learns to skip layers; a survey of such dynamic networks is given in [20]. Model-level selection is closest to this work: Bolukbasi *et al.* [7] learn a policy that decides whether to evaluate a more expensive network, IDK cascades [6] learn when a model should say "I don't know", and NoScope [21] cascades cheap specialised models in front of an expensive detector for video. FrugalML [22] and FrugalGPT [23] learn which paid prediction API to call under a monetary budget. Neurosurgeon [24] and Edgent [25] adapt by partitioning a single network between device and cloud according to network conditions. These methods optimise an accuracy-cost trade-off but treat all errors as equally costly and, except for the edge-partitioning works, do not condition on a changing device state.

D. Selective prediction, deferral and calibration

Selective classification lets a model abstain on inputs where it is likely to be wrong [26]; learning-to-defer frameworks train a classifier jointly with a rejector that hands inputs to a human expert [27], [28]. Such decisions rely on well-calibrated confidence, which modern networks often lack [29]; label smoothing [30], [31] is a simple way to soften over-confident outputs, and maximum softmax probability is a common baseline for detecting misclassified or out-of-distribution inputs [32]. Cost-sensitive learning [33] formalises unequal misclassification costs. These lines of work handle abstention or unequal costs, but not jointly with multi-model routing under resource constraints.

E. Research gap

Table I summarises the survey. To the best of our knowledge, no prior work on satellite damage assessment combines (i) learned selection among several models, (ii) conditioning on the current device resources, (iii) asymmetric safety costs, and (iv) deferral and human-review actions in a single controller. ResQRoute targets exactly this combination.

TABLE I. COMPARISON OF RELATED APPROACHES

Approach	Learned model choice	Device resources	Asym. risk	Defer / human
xBD CNNs [8], [9], [10], [11]	No	No	No	No
Fixed-threshold cascade [5]	No	No	No	No
Early-exit nets [16], [17], [18]	Partly	No	No	No
Adaptive nets / IDK [6], [7]	Yes	No	No	Partly
NoScope, FrugalML [21], [22]	Yes	Partly (budget)	No	No
Neurosurgeon, Edgent [24], [25]	No	Yes	No	No
Learning to defer [27], [28]	No	No	Partly	Yes
ResQRoute (ours)	Yes	Yes	Yes	Yes

III. PROPOSED METHODOLOGY

A. System overview

ResQRoute contains three damage classifiers of increasing cost and a routing controller. The pipeline is small model → controller → action: for each building crop the small model always runs first. The controller then observes the small model's output, image-quality features and the device state, and selects one of five actions $a \in$ {accept, escalate-EfficientNet, escalate-ResNet-18, defer, human-review}. Escalation targets are ordered by their *measured* CPU cost rather than by nominal size (Section V).

B. Data preparation

The labelled training release of xBD [2] (2,799 pre/post image pairs of 1024 × 1024 pixels from 10 events) was indexed into 162,787 building polygons. Because the minor-damage class is rare in most events, three events were selected that together cover all damage levels: Hurricane Michael and Hurricane Matthew (wind damage of all severities, most minor-damage examples) and the Santa Rosa wildfire (a second hazard type with many destroyed buildings). Hurricane Harvey was excluded because its many major-damage labels stem from flooding that is largely invisible from above. Major damage and destroyed were merged into *severe damage*, giving three classes; unclassified buildings and polygons smaller than 150 px² were removed.

From 43,148 usable buildings, 2,000 per class were sampled (severe split equally between major and destroyed), yielding 6,000 buildings from 668 tiles. For each, the polygon's bounding box was enlarged by 20% per side (minimum 32 px), cropped from both the pre- and post-event image, and letterboxed to 224 × 224. The data were split 70/15/15 (4,299 / 849 / 852) with stratification by class **and grouping by source tile**, so that neighbouring buildings from the same scene never appear in different splits.

C. Base classifiers

MobileNetV3-Small [12], ResNet-18 [13] and EfficientNet-B0 [14], pretrained on ImageNet [15], were fine-tuned on the post-event crop with PyTorch [34]:

AdamW (learning rate 3×10^{-4}), cosine schedule, flips, 90° rotations and colour jitter, class-weighted cross-entropy with label smoothing 0.1 [30], [31] to temper over-confident probabilities that the controller consumes, and checkpoint selection by validation macro-F1 (early stopping after 6 epochs without improvement, maximum 20).

D. Controller features

At decision time the controller sees only information that is actually available before escalation:

- **Uncertainty:** the small model's class probabilities, maximum confidence, top-2 margin and predictive entropy $H = -\Sigma\, p_k \log p_k$.
- **Image quality:** variance of the Laplacian (blur), mean and standard deviation of grey level (brightness, contrast), fraction of the crop that is image rather than padding (building size), mean absolute pre/post difference (change magnitude), and the shares of near-black and near-white pixels (shadow, cloud or smoke proxies).
- **Device state:** latency budget, free memory, network availability, battery level, queue length, and whether each larger model fits the current budget and memory.

E. Oracle routes and asymmetric safety

Training labels are derived from the logged behaviour of the three models on each crop i under device state s. Let c_m be whether model m is correct and $f_m(s)$ whether it fits the budget and memory. The oracle route is:

$$a^*(i,s) = \text{accept if } c_small;\ \text{else the cheapest } m \text{ with } c_m \wedge f_m(s);\ \text{else defer if } \exists m{:}\ c_m;\ \text{else human-review} \quad (1)$$

with human review replaced by deferral when the network is unavailable. To encode the asymmetric cost of errors [33], each training example is weighted by w = 1 for non-severe buildings, w = 3 for severe buildings, and w = 5 for severe buildings that the small model misclassifies. The controller is a histogram-based gradient-boosted tree ensemble [35], [36] from scikit-learn [37]. At inference two hard guards apply: a model that does not fit the device is never chosen (the request is deferred instead), and human review is converted to deferral when offline.

F. Deferred escalation queue

Deferred buildings are re-processed by EfficientNet-B0 when resources return, in order of the priority

$$P_i = U_i \times R_i \times D_i \times A_i \quad (2)$$

where U_i is the normalised entropy (uncertainty), R_i the small model's probability of severe damage (risk of a dangerous miss), $D_i = 0.5\cdot p_minor + p_severe$ the expected severity, and $A_i = 0.5 + 0.5\cdot$(building size fraction) an area-importance term.

IV. EXPERIMENTAL SETUP

Training ran on Kaggle's free CPU sessions (4 cores). All inference measurements were made on a laptop CPU with one thread, batch size 1, after 10 warm-up passes; each crop's latency is the median of three timed forward passes, and memory is the peak resident-set increase for loading a model and running one forward pass, measured in a fresh process. The controller was trained on the 849 validation crops (on which the base models were not trained, so their outputs are honest), each paired with 8 random device states, and evaluated on the 852 test crops × 8 states = 6,816 decisions. Device states draw the latency budget from {60, 180, 400} ms, free memory from {150, 250, 500} MB, network availability with probability 0.7, and battery uniformly in [0.05, 1]; below 20% battery the budget is halved.

Baselines: always running each single model; a **fixed-confidence cascade** (accept when confidence ≥ t, else escalate, ignoring resources); and a **resource-aware cascade without risk** (accept when confident, else the cheapest model that fits, else defer). Both thresholds were tuned on validation (t = 0.74 and t = 0.52). Metrics: accuracy, macro-F1, severe-damage recall, dangerous false-negative rate (truly severe predicted as no-damage), average latency and peak memory, the share of decisions exceeding the device budget, and the distribution of actions. Human-reviewed buildings are counted as correctly labelled; results without this credit are also reported. Confidence intervals use a crop-level bootstrap [38] with 500 resamples.

V. RESULTS AND DISCUSSION

A. Base classifiers

TABLE II. BASE CLASSIFIERS ON THE TEST SPLIT (852 CROPS)

Model	Acc.	Macro-F1	Severe recall	CPU ms	Mem. MB
MobileNetV3-Small	0.604	0.597	0.398	22.9	98.5
ResNet-18	0.622	0.610	0.377	219.8	172.1
EfficientNet-B0	0.617	0.610	0.419	105.3	121.5

Table II shows that the three models are close in accuracy (validation macro-F1 0.776, 0.795 and 0.778), while their costs differ by an order of magnitude. On a CPU, ResNet-18 is the slowest and most memory-hungry model despite not being the most accurate, so cost cannot be inferred from nominal model size. The dominant error for all models is confusing severe with minor damage. At least one of the three models is correct on 75.7% of test crops versus 62.2% for the best single model, which indicates substantial room for routing.

B. Routing performance

TABLE III. ROUTING POLICIES ON THE TEST SPLIT (6,816 DECISIONS)

Policy	Macro-F1	Dang. FN %	Avg ms	Over budget %
Always MobileNetV3-S	0.597	6.3	23	0.5
Always EfficientNet-B0	0.610	4.2	126	37.0
Always ResNet-18	0.610	9.5	235	78.3
Fixed-confidence cascade	0.612	7.4	113	32.5
Resource-aware, no risk	0.605	4.6	33	0.4
ResQRoute (ours)	**0.631**	**4.6**	**27**	**0.5**

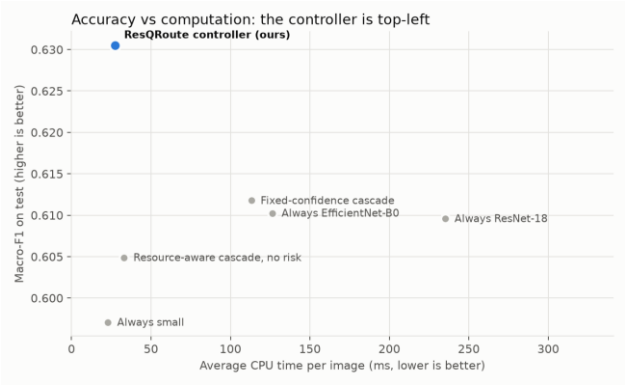


Fig. 1. Macro-F1 versus average CPU time per image on the test split.

Table III and Fig. 1 show the central result. The controller attains the highest macro-F1 (0.631) at an average of 27 ms per image, about one-fifth of always running EfficientNet-B0 and one-ninth of ResNet-18. It respects the device in 99.5% of decisions, whereas the fixed-confidence cascade exceeds the budget or memory in 32.5% of them, because it escalates regardless of resources. Its dangerous false-negative rate (4.6%) is lower than that of the small model (6.3%) and the fixed cascade (7.4%), and close to the best single model (4.2%). The controller accepted the small model in 82.8% of decisions, escalated in 4.0%, deferred 9.1% and sent 4.0% to human review.

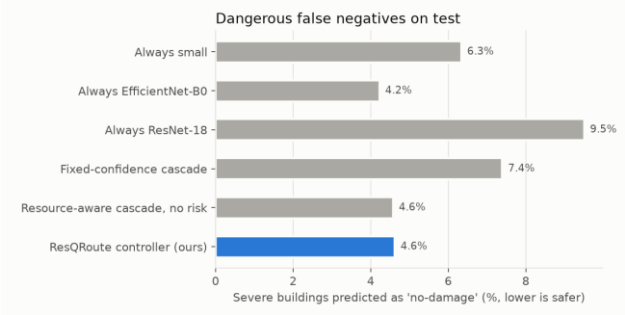


Fig. 2. Share of truly severe buildings predicted as no-damage (lower is safer).

C. Adaptation to resources

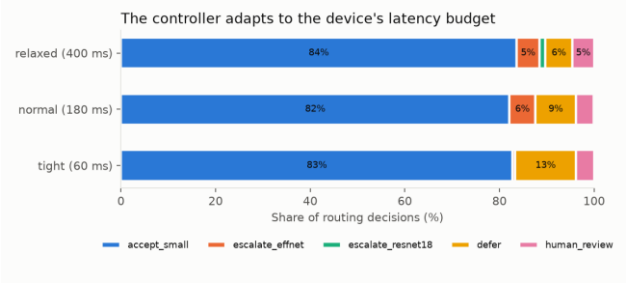


Fig. 3. Controller actions by latency budget.

Fig. 3 shows the controller's behaviour changing with the device: under a tight 60 ms budget it almost never escalates (0.5%) and defers 12.9% of buildings; under a relaxed 400 ms budget it escalates 6.1% and defers only 5.7%. Accuracy rises accordingly from 0.623 to 0.652 while average latency stays between 23 and 31 ms.

D. Deferred queue and learned features

Of the 619 deferred decisions, 43.7% of those in the top half of the priority queue were truly severe, against 34.5% in the bottom half, so equation (2) brings dangerous buildings forward. Permutation importance on the validation set ranks network availability, the small model's probability of severe damage, the top-2 margin, whether each larger model fits the budget, and the dark-pixel share and brightness of the crop as the most influential features, confirming that the controller combines uncertainty, resources and image quality.

E. Statistical robustness

A crop-level bootstrap gives a 95% confidence interval of [−0.003, +0.044] for the macro-F1 difference between ResQRoute and always running EfficientNet-B0, and [−0.006, +0.043] against the fixed-confidence cascade. Without crediting human review, ResQRoute's macro-F1 is 0.607 versus 0.610 for EfficientNet-B0 (difference −0.003, 95% CI [−0.026, +0.019]). The defensible conclusion is therefore that ResQRoute **matches** the accuracy of the large models at a fraction of their cost while staying within the device budget and reducing dangerous errors relative to a conventional cascade; the accuracy gain itself is mostly due to human review and is not statistically significant on this test set.

VI. LIMITATIONS

- **Simulated device states.** Model latencies and memory are measured, but budgets, network and battery are sampled from fixed distributions; field traces would strengthen the evaluation.
- **Small controller training set.** Only 849 held-out crops are available for the controller; out-of-fold predictions over the training split would provide about five times more.
- **Split difficulty.** The test split contains half of its severe buildings from Hurricane Matthew, whose

imagery is the blurriest; test macro-F1 (≈0.61) is therefore well below validation (≈0.78).

- **Post-image only.** The base classifiers use only the post-event crop; xView2 solutions show that comparing pre- and post-event images substantially improves damage grading [9].
- **Human accuracy assumed perfect.** Human-reviewed cases are counted as correct.

VII. CONCLUSION AND FUTURE WORK

This paper presented ResQRoute, a learned controller that decides, per building, whether to accept a lightweight damage prediction, escalate to a larger model, defer, or request human review, using uncertainty, image quality, device resources and asymmetric safety costs. On xBD it matched the macro-F1 of always running a large model at one-fifth to one-ninth of the CPU time, stayed within the device budget in 99.5% of decisions, and reduced dangerous severe-to-no-damage errors compared with a fixed-confidence cascade. Future work will add pre/post fusion to the base models, train the controller on out-of-fold predictions, evaluate on real device and network traces and on unseen disaster events, and add an out-of-distribution detector so that unfamiliar regions are routed to human review rather than trusted on confidence alone [32].

Data availability: xBD is available under CC BY-NC-SA 4.0. Code, crop manifests, model weights and logs: https://github.com/SriharshithaAnkareddy25/ResQRoute-

VIII. REFERENCES

[1] L. Dong and J. Shan, "A comprehensive review of earthquake-induced building damage detection with remote sensing techniques," ISPRS J. Photogramm. Remote Sens., vol. 84, pp. 85-99, 2013.

[2] R. Gupta et al., "xBD: A dataset for assessing building damage from satellite imagery," arXiv:1911.09296, 2019.

[3] R. Gupta et al., "Creating xBD: A dataset for assessing building damage from satellite imagery," in Proc. IEEE/CVF CVPR Workshops, 2019.

[4] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, and J. Zhang, "Edge intelligence: Paving the last mile of artificial intelligence with edge computing," Proc. IEEE, vol. 107, no. 8, pp. 1738-1762, 2019.

[5] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in Proc. IEEE CVPR, 2001.

[6] X. Wang, Y. Luo, D. Crankshaw, A. Tumanov, F. Yu, and J. E. Gonzalez, "IDK cascades: Fast deep learning by learning not to overthink," in Proc. UAI, 2018.

[7] T. Bolukbasi, J. Wang, O. Dekel, and V. Saligrama, "Adaptive neural networks for efficient inference," in Proc. ICML, 2017.

[8] J. Z. Xu, W. Lu, Z. Li, P. Khaitan, and V. Zaytseva, "Building damage detection in satellite imagery using convolutional neural networks," arXiv:1910.06444, 2019.

[9] E. Weber and H. Kane, "Building disaster damage assessment in satellite imagery with multi-temporal fusion," arXiv:2004.05525, 2020.

[10] R. Gupta and M. Shah, "RescueNet: Joint building segmentation and damage assessment from satellite imagery," in Proc. ICPR, 2020.

[11] Y. Shen et al., "BDANet: Multiscale convolutional neural network with cross-directional attention for building damage assessment from satellite images," IEEE Trans. Geosci. Remote Sens., vol. 60, 2022.

[12] A. Howard et al., "Searching for MobileNetV3," in Proc. IEEE/CVF ICCV, 2019.

[13] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in Proc. IEEE CVPR, 2016.

[14] M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for convolutional neural networks," in Proc. ICML, 2019.

[15] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in Proc. IEEE CVPR, 2009.

[16] S. Teerapittayanon, B. McDanel, and H. T. Kung, "BranchyNet: Fast inference via early exiting from deep neural networks," in Proc. ICPR, 2016.

[17] G. Huang, D. Chen, T. Li, F. Wu, L. van der Maaten, and K. Q. Weinberger, "Multi-scale dense networks for resource efficient image classification," in Proc. ICLR, 2018.

[18] Y. Kaya, S. Hong, and T. Dumitras, "Shallow-deep networks: Understanding and mitigating network overthinking," in Proc. ICML, 2019.

[19] X. Wang, F. Yu, Z.-Y. Dou, T. Darrell, and J. E. Gonzalez, "SkipNet: Learning dynamic routing in convolutional networks," in Proc. ECCV, 2018.

[20] Y. Han, G. Huang, S. Song, L. Yang, H. Wang, and Y. Wang, "Dynamic neural networks: A survey," IEEE Trans. Pattern Anal. Mach. Intell., vol. 44, no. 11, pp. 7436-7456, 2022.

[21] D. Kang, J. Emmons, F. Abuzaid, P. Bailis, and M. Zaharia, "NoScope: Optimizing neural network queries over video at scale," Proc. VLDB Endow., vol. 10, no. 11, 2017.

[22] L. Chen, M. Zaharia, and J. Zou, "FrugalML: How to use ML prediction APIs more accurately and cheaply," in Proc. NeurIPS, 2020.

[23] L. Chen, M. Zaharia, and J. Zou, "FrugalGPT: How to use large language models while reducing cost and improving performance," arXiv:2305.05176, 2023.

[24] Y. Kang et al., "Neurosurgeon: Collaborative intelligence between the cloud and mobile edge," in Proc. ASPLOS, 2017.

[25] E. Li, Z. Zhou, and X. Chen, "Edge intelligence: On-demand deep learning model co-inference with device-edge synergy," in Proc. ACM SIGCOMM Workshop on Mobile Edge Communications, 2018.

[26] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," in Proc. NeurIPS, 2017.

[27] D. Madras, T. Pitassi, and R. Zemel, "Predict responsibly: Improving fairness and accuracy by learning to defer," in Proc. NeurIPS, 2018.

[28] H. Mozannar and D. Sontag, "Consistent estimators for learning to defer to an expert," in Proc. ICML, 2020.

[29] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in Proc. ICML, 2017.

[30] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in Proc. IEEE CVPR, 2016.

[31] R. Muller, S. Kornblith, and G. Hinton, "When does label smoothing help?," in Proc. NeurIPS, 2019.

[32] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," in Proc. ICLR, 2017.

[33] C. Elkan, "The foundations of cost-sensitive learning," in Proc. IJCAI, 2001.

[34] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Proc. NeurIPS, 2019.

[35] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Ann. Statist., vol. 29, no. 5, pp. 1189-1232, 2001.

[36] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in Proc. NeurIPS, 2017.

[37] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825-2830, 2011.

[38] B. Efron and R. J. Tibshirani, An Introduction to the Bootstrap. New York, NY, USA: Chapman & Hall, 1993.